СПРАВОЧНИК • BEVY 0.20

ОБНОВЛЁН 26.10.2026



Bevy 0.20
справочник на русском

Только то, что нужно для ParodiaR: приложение и плагины, ECS, порядок систем, сообщения, ввод, камера, свой меш, материалы и шейдеры WESL, фоновые задачи, состояния, интерфейс. Имена в коде — в оригинале, объяснения — по-русски. Примеры собраны по официальным примерам Bevy v0.20.0.

ECS	Ввод	Меш	Шейдеры	Задачи
системы, запросы	камера	чанк на экране	WESL	состояния, UI

Ведёт Claude вместе с Даней • исходник: docs/bevy_ru.md

Содержание

01 Как читать справочник

02 Приложение: App и плагины

03 ECS: сущности, компоненты, системы

04 Порядок систем

05 Сообщения: Message

06 Ввод: клавиатура и мышь

07 Камера и Transform

08 Свой меш: чанк на экране

09 Свой материал и шейдер (WESL)

10 Фоновые задачи

11 Состояния игры

12 Текст, F3 и рамка блока

13 Ассеты: файлы игры

14 Частые ошибки

15 Словарь: имя в коде → что значит

01 Как читать справочник

> Это короткий справочник по **Bevy 0.20** на русском — только то, что нужно для ParodiaR по ТЗ. Каждый пример собран по официальным примерам Bevy именно версии 0.20 (папка `examples` репозитория, метка `v0.20.0`), а не по памяти и не по старым урокам из интернета.

ВЕРСИЯ	ШЕЙДЕРЫ	СООБЩЕНИЯ	ПРОВЕРКА
Bevy 0.20.0 • Rust 1.99	**WESL (.wesl)**	**Message, не Event**	**cargo doc --open**

Правило имён

Что	Как пишется здесь
Код: типы, функции, методы, поля	**всегда в оригинале**, моноширинным шрифтом: `Query`, `Commands`, `spawn`
Первое упоминание	имя + перевод в скобках: `Query` (запрос к сущностям)
Объяснения	по-русски
Комментарии в примерах	по-русски — их можно оставлять в своём коде

Имена в коде нельзя переводить: компилятор знает только `Transform`, а не «Преобразование». Если читаешь английскую страницу через переводчик браузера — **код сверяй с оригиналом**, переводчик иногда «переводит» и его.

Как проверить, что пример верный

- **Документация твоей версии без интернета:** в папке проекта выполнить

```
cd D:\Program\rust\ParodiaR
cargo doc --open
```

Откроется браузер со всеми типами Bevy 0.20. Слева поиск: вводишь `Query` — видишь все его методы. - **В VS Code:** навести мышь на имя — всплывёт описание; `F12` — перейти к исходнику Bevy. - **Старые уроки:** если в примере из интернета видишь `EventReader`, `#import`, `.wgsl` с `#import`, `Camera3dBundle`, `PbrBundle` — это **старая** версия Bevy. В 0.20 так не работает.

Важно: примеры не проверены компилятором здесь. Я собирал их из официальных примеров Bevy 0.20 и сверял имена с исходниками движка, но собрать в своей среде не могу (нет доступа к crates.io). Если при `cargo run` будет ошибка — пришли её текст: обычно это одна строка (забытый `use` или опечатка).

02 Приложение: App и плагины

> `App` (приложение) — это вся игра. В `main.rs` её собирают как конструктор: добавляют плагины, ресурсы и системы, потом вызывают `run()`.

```rust
use bevy::prelude::*; // prelude — «всё самое нужное» одной строкой

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)            // окно, ввод, рендер, звук, ассеты...
        .add_systems(Startup, hello)            // один раз при старте
        .add_systems(Update, every_frame)       // каждый кадр
        .run();
}

fn hello() {
    info!("Игра запущена"); // info! — запись в лог (видно в консоли)
}

fn every_frame() {}
```

Расписания (Schedule — «когда запускать»)

Расписание	Когда	Для чего в ParodiaR
`Startup`	один раз в начале	создать камеру, загрузить реестр блоков
`Update`	каждый кадр	ввод, камера, подгрузка чанков, интерфейс
`FixedUpdate`	постоянный шаг (по умолчанию 64 раза/с)	физика игрока, вода, мобы
`Last`	в конце каждого кадра	редкие служебные дела
`OnEnter(Состояние)` / `OnExit(...)`	при входе/выходе из состояния	меню, загрузка мира (раздел 11)

Свой плагин

`Plugin` (плагин) — кусок приложения, который сам регистрирует свои системы и ресурсы. В ТЗ: одна фича — один плагин — одна папка.

```rust
// crates/parodia_game/src/debug/mod.rs
use bevy::prelude::*;

pub struct DebugPlugin;

impl Plugin for DebugPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Startup, spawn_debug_text)
            .add_systems(Update, update_debug_text);
    }
}

fn spawn_debug_text() {}
fn update_debug_text() {}
```

```rust
// main.rs
mod debug;

fn main() {
    App::new()
        .add_plugins((DefaultPlugins, debug::DebugPlugin)) // несколько — кортежем в скобках
        .run();
}
```

Настройки окна и пиксельные текстуры

`DefaultPlugins` (стандартный набор плагинов) можно донастроить через `.set(...)`:

```rust
use bevy::{prelude::*, window::PresentMode};

fn main() {
    App::new()
        .add_plugins(
            DefaultPlugins
                .set(WindowPlugin {
                    primary_window: Some(Window {
                        title: "ParodiaR".into(),
                        resolution: (1280, 720).into(),
                        present_mode: PresentMode::AutoVsync, // вертикальная синхронизация
                        ..default()
                    }),
                    ..default()
                })
                // текстуры без размытия — для пиксель-арта обязательно
                .set(ImagePlugin::default_nearest()),
        )
        .run();
}
```

- `..default()` — «остальные поля по умолчанию». Встречается в Bevy везде.

- `ImagePlugin::default_nearest()` — все текстуры по умолчанию с фильтром «ближайший пиксель» (nearest), без размытия.

03 ECS: сущности, компоненты, системы

> ECS (Entity Component System) — способ хранить данные игры. **Сущность** — просто номер. **Компонент** — данные, прикреплённые к сущности. **Система** — обычная функция, которая обрабатывает все сущности с нужными компонентами. **Ресурс** — одна общая вещь на всю игру.

Компонент и ресурс

```rust
use bevy::prelude::*;

#[derive(Component)]          // Component — «это можно прикрепить к сущности»
struct Player;                // компонент-метка: без данных, просто «это игрок»

#[derive(Component)]
struct Velocity(Vec3);        // компонент с данными

#[derive(Resource, Default)]  // Resource — «одна штука на всю игру»
struct WorldSettings {
    seed: u64,
    view_distance: u32,
}
```

Ресурс добавляется в приложение:

```rust
app.init_resource::<WorldSettings>();            // через Default
app.insert_resource(WorldSettings { seed: 42, view_distance: 8 }); // своим значением
```

Создать сущность: Commands

`Commands` (команды) — очередь изменений мира. Сущности создаются и удаляются **не сразу**, а между системами (это нужно, чтобы системы могли работать параллельно).

```rust
fn spawn_player(mut commands: Commands) {
    commands.spawn((              // в скобках — набор компонентов (кортеж)
        Player,
        Velocity(Vec3::ZERO),
        Transform::from_xyz(0.0, 70.0, 0.0),
    ));
}
```

- `commands.spawn(...)` — создать; возвращает `EntityCommands`, у которого `.id()` — номер сущности.
- `commands.entity(id).insert(Компонент)` — добавить компонент; `.remove::<Компонент>()` — убрать; `.despawn()` — удалить сущность вместе с детьми.
- Дети (иерархия): макрос `children![...]` — например, камера внутри игрока:

```rust
commands.spawn((
    Player,
    Transform::from_xyz(0.0, 70.0, 0.0),
    Visibility::default(),
    children![(
        Camera3d::default(),
        Transform::from_xyz(0.0, 1.62, 0.0), // глаза на высоте 1,62 м
    )],
));
```

Читать и менять: Query, Res, ResMut

```rust
// Query — запрос: «дай все сущности, у которых есть Transform И Velocity»
fn apply_velocity(time: Res<Time>, mut query: Query<(&mut Transform, &Velocity)>) {
    for (mut transform, velocity) in &mut query {
        transform.translation += velocity.0 * time.delta_secs();
    }
}
```

Параметр системы	Что значит
`Query<&T>`	читать компонент `T` у всех сущностей, где он есть
`Query<&mut T>`	менять компонент (перебор — `for x in &mut query`)
`Query<&T, With<Player>>`	только у сущностей, где **есть** `Player` (сам `Player` не читаем)
`Query<&T, Without<Player>>`	только где `Player` **нет**
`Query<&T, Changed<T>>`	только где `T` изменился с прошлого запуска системы
`Query<(Entity, &T)>`	вместе с номером сущности
`Single<&T, With<Player>>`	ровно **одна** такая сущность (игрок, камера). Если её нет или их две — система просто не запустится
`Res<R>` / `ResMut<R>`	прочитать / изменить ресурс
`Local<T>`	«своя» переменная системы, живёт между кадрами (счётчик, таймер)
`Commands`	создать/удалить сущности, добавить ресурсы

```rust
fn player_height(player: Single<&Transform, With<Player>>) {
    info!("Игрок на высоте {}", player.translation.y);
}
```

Важно: два `&mut` одного компонента в одной системе — ошибка. `Query<&mut Transform, With<Player>>` и `Query<&mut Transform, With<Camera3d>>` в одной функции Bevy не пустит: вдруг это одна сущность. Решение — добавить `Without<...>` ко второму запросу, чтобы наборы точно не пересекались: `Query<&mut Transform, (With<Camera3d>, Without<Player>)>`.

Время

Что	Код
сколько секунд прошло с прошлого кадра	`time.delta_secs()` (параметр `time: Res<Time>`)
сколько секунд прошло с запуска	`time.elapsed_secs()`
таймер «раз в секунду»	`Timer::from_seconds(1.0, TimerMode::Repeating)`, каждый кадр `timer.tick(time.delta())`, проверка `timer.just_finished()`

Движение умножается на `delta_secs()` — тогда скорость одинаковая на 30 и на 144 кадрах. **Мышь — нет** (см. раздел 06).

04 Порядок систем

> По умолчанию системы одного расписания идут **параллельно и в любом порядке**. Это быстро, но если «сначала ввод, потом движение» важно — порядок задают явно.

Цепочка и before/after

```
app.add_systems(Update, (read_input, move_player, move_camera).chain()); // строго по очереди
app.add_systems(Update, move_camera.after(move_player));                 // одно после другого
```

Группы систем: SystemSet

Для большого проекта удобнее группы (как в ТЗ, файл `sets.rs`):

```
// crates/parodia_game/src/sets.rs
use bevy::prelude::*;

#[derive(SystemSet, Debug, Clone, PartialEq, Eq, Hash)]
pub enum GameSet {
    Input,
    Player,
    WorldEdit,
    Streaming,
    Meshing,
    Camera,
    Ui,
}

pub fn configure(app: &mut App) {
    app.configure_sets(
        Update,
        (
            GameSet::Input,
            GameSet::Player,
            GameSet::WorldEdit,
            GameSet::Streaming,
            GameSet::Meshing,
            GameSet::Camera,
            GameSet::Ui,
        )
            .chain(), // группы идут строго в этом порядке
    );
}
```

Плагин кладёт свою систему в группу:

```
app.add_systems(Update, update_chunk_meshes.in_set(GameSet::Meshing));
```

Условия запуска: run_if

```
use bevy::input::common_conditions::input_just_pressed;

app.add_systems(Update, toggle_debug.run_if(input_just_pressed(KeyCode::F3)));
app.add_systems(Update, move_player.run_if(in_state(AppState::InGame))); // раздел 11
```

Постоянный шаг: FixedUpdate

```
app.insert_resource(Time::<Fixed>::from_hz(30.0)) // 30 шагов в секунду (по ТЗ)
    .add_systems(FixedUpdate, player_physics);
```

Внутри системы `FixedUpdate` обычный `Res<Time>` уже показывает фиксированный шаг: `time.delta_secs()` = 1/30. Чтобы движение на экране было плавным, позицию для картинки интерполируют между двумя шагами — сдвиг до следующего шага даёт `Res<Time<Fixed>>` → `overstep_fraction()`.

05 Сообщения: Message

> Сообщение — «что-то случилось» (блок сломан, игрок приземлился). Одна система пишет, любые другие читают. В Bevy 0.17+ буферные события называются **Message**. Старое `EventReader` из уроков — это прошлые версии.

```rust
use bevy::prelude::*;

#[derive(Message, Debug, Clone, Copy)]
pub struct BlockChanged {
    pub pos: IVec3,   // IVec3 — вектор из трёх i32
    pub old: u16,
    pub new: u16,
}

// регистрация — обязательно, иначе паника при запуске
app.add_message::<BlockChanged>();

// писать
fn break_block(mut writer: MessageWriter<BlockChanged>) {
    writer.write(BlockChanged { pos: IVec3::new(1, 64, 2), old: 1, new: 0 });
}

// читать (каждая читающая система получает все сообщения сама)
fn remesh_on_change(mut reader: MessageReader<BlockChanged>) {
    for msg in reader.read() {
        info!("Блок {:?} изменился: {} → {}", msg.pos, msg.old, msg.new);
    }
}
```

- Чтобы читатель увидел сообщение **в том же кадре**, он должен стоять **после** писателя (`.chain()` или `SystemSet`). Иначе — на кадр позже, что тоже часто нормально.
- Сообщения живут примерно 2 кадра: кто не прочитал — пропустил.
- Для «реакции сразу на событие у конкретной сущности» в Bevy есть ещё **наблюдатели** (observers, `add_observer`, `On<...>`). На первых этапах ТЗ хватит `Message`.

06 Ввод: клавиатура и мышь

Клавиатура

```rust
fn keyboard(keys: Res<ButtonInput<KeyCode>>) {
    if keys.pressed(KeyCode::KeyW) { /* держат */ }
    if keys.just_pressed(KeyCode::F5) { /* нажали в этом кадре */ }
    if keys.just_released(KeyCode::Space) { /* отпустили в этом кадре */ }
}
```

`KeyCode` **— это физическое место клавиши**, а не буква. `KeyCode::KeyW` — клавиша, где на английской раскладке W, а на русской — Ц. Поэтому WASD работают при любой раскладке — для тебя это важно. Если нужен именно символ (например «?» для справки), используется `Res<ButtonInput<Key>>` с `Key::Character("?".into())`.

Нужные имена клавиш: `KeyCode::KeyA` … `KeyZ`, `Digit1` … `Digit9`, `Space`, `ShiftLeft`, `ControlLeft`, `Escape`, `Tab`, `F1` … `F12`, `ArrowUp`.

Кнопки мыши

```rust
fn mouse_buttons(mouse: Res<ButtonInput<MouseButton>>) {
    if mouse.pressed(MouseButton::Left) { /* ломать */ }
    if mouse.just_pressed(MouseButton::Right) { /* ставить */ }
}
```

Движение мыши и колесо

```rust
use bevy::input::mouse::{AccumulatedMouseMotion, AccumulatedMouseScroll};

fn mouse_move(motion: Res<AccumulatedMouseMotion>, scroll: Res<AccumulatedMouseScroll>) {
    let d = motion.delta;        // Vec2: сдвиг мыши за кадр (в пикселях)
    let wheel = scroll.delta.y;  // колесо: + вверх, − вниз
}
```

Мышь не умножают на `delta_secs()`: `delta` — уже весь сдвиг за кадр. Если умножить, поворот камеры будет зависеть от FPS.

Захват курсора

```rust
use bevy::window::{CursorGrabMode, CursorOptions};

fn grab_mouse(
    mut cursor: Single<&mut CursorOptions>,
    mouse: Res<ButtonInput<MouseButton>>,
    keys: Res<ButtonInput<KeyCode>>,
) {
    if mouse.just_pressed(MouseButton::Left) {
        cursor.visible = false;
        cursor.grab_mode = CursorGrabMode::Locked;   // курсор «заперт» в центре
    }
    if keys.just_pressed(KeyCode::Escape) {
        cursor.visible = true;
        cursor.grab_mode = CursorGrabMode::None;     // отпустить
    }
}
```

07 Камера и Transform

> `Transform` (положение) — позиция, поворот и масштаб сущности. `Camera3d` — 3D-камера. Оси в Bevy: **Y — вверх**, «вперёд» камеры — **−Z**, 1 единица = 1 метр.

Код	Что делает
`Transform::from_xyz(x, y, z)`	позиция
`.looking_at(цель, Vec3::Y)`	повернуть «лицом» к точке
`transform.translation`	позиция (`Vec3`), можно менять
`transform.rotation`	поворот (`Quat` — кватернион)
`transform.forward()` , `.right()` , `.up()`	направления «вперёд/вправо/вверх» (тип `Dir3` , как `Vec3` через `*`)
`Quat::from_euler(EulerRot::YXZ, рыскание, тангаж, 0.0)`	поворот из углов (в радианах)
`30.0_f32.to_radians()`	градусы → радианы

Полный пример: камера-наблюдатель (этап 1 ТЗ)

Это готовый файл для этапа 1: полёт WASD, Пробел — вверх, Shift — вниз, Ctrl — быстрее, мышь — обзор, ЛКМ — захватить мышь, Esc — отпустить. Собран по примерам `camera/first_person_view_model.rs` и `input/mouse_grab.rs` из Bevy 0.20.

```rust
// crates/parodia_game/src/player/fly_camera.rs
use std::f32::consts::FRAC_PI_2;

use bevy::{
    input::mouse::AccumulatedMouseMotion,
    prelude::*,
    window::{CursorGrabMode, CursorOptions},
};

/// Камера, которая летает свободно (режим наблюдателя).
#[derive(Component)]
pub struct FlyCamera {
    pub speed: f32,        // метров в секунду
    pub sensitivity: Vec2, // чувствительность мыши по X и Y
}

pub struct FlyCameraPlugin;

impl Plugin for FlyCameraPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Startup, spawn_camera)
            .add_systems(Update, (grab_mouse, look_around, fly).chain());
    }
}

fn spawn_camera(mut commands: Commands) {
    commands.spawn((
        Camera3d::default(),
        Transform::from_xyz(0.0, 3.0, 8.0).looking_at(Vec3::ZERO, Vec3::Y),
        FlyCamera { speed: 10.0, sensitivity: Vec2::new(0.003, 0.002) },
    ));
}

fn grab_mouse(
    mut cursor: Single<&mut CursorOptions>,
    mouse: Res<ButtonInput<MouseButton>>,
    keys: Res<ButtonInput<KeyCode>>,
) {
    if mouse.just_pressed(MouseButton::Left) {
        cursor.visible = false;
        cursor.grab_mode = CursorGrabMode::Locked;
    }
    if keys.just_pressed(KeyCode::Escape) {
        cursor.visible = true;
        cursor.grab_mode = CursorGrabMode::None;
    }
}

fn look_around(
    motion: Res<AccumulatedMouseMotion>,
    cursor: Single<&CursorOptions>,
    camera: Single<(&mut Transform, &FlyCamera)>,
) {
    // мышь не захвачена — не вращаем (можно кликать по окну)
    if cursor.grab_mode == CursorGrabMode::None || motion.delta == Vec2::ZERO {
```

```
        return;
    }
    let (mut transform, fly) = camera.into_inner();

    let (yaw, pitch, roll) = transform.rotation.to_euler(EulerRot::YXZ);
    let yaw = yaw - motion.delta.x * fly.sensitivity.x;
    // не даём смотреть ровно вверх/вниз — там поворот «ломается»
    const LIMIT: f32 = FRAC_PI_2 - 0.01;
    let pitch = (pitch - motion.delta.y * fly.sensitivity.y).clamp(-LIMIT, LIMIT);
    transform.rotation = Quat::from_euler(EulerRot::YXZ, yaw, pitch, roll);
}

fn fly(
    keys: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    camera: Single<(&mut Transform, &FlyCamera)>,
) {
    let (mut transform, fly) = camera.into_inner();

    // «вперёд» без наклона: летим горизонтально, куда смотрим
    let forward = Vec3::new(transform.forward().x, 0.0,
transform.forward().z).normalize_or_zero();
    let right = Vec3::new(transform.right().x, 0.0, transform.right().z).normalize_or_zero();

    let mut dir = Vec3::ZERO;
    if keys.pressed(KeyCode::KeyW) { dir += forward; }
    if keys.pressed(KeyCode::KeyS) { dir -= forward; }
    if keys.pressed(KeyCode::KeyD) { dir += right; }
    if keys.pressed(KeyCode::KeyA) { dir -= right; }
    if keys.pressed(KeyCode::Space) { dir += Vec3::Y; }
    if keys.pressed(KeyCode::ShiftLeft) { dir -= Vec3::Y; }

    let boost = if keys.pressed(KeyCode::ControlLeft) { 4.0 } else { 1.0 };
    transform.translation += dir.normalize_or_zero() * fly.speed * boost * time.delta_secs();
}
```

Угол обзора и зум колесом

```
use bevy::input::mouse::AccumulatedMouseScroll;

fn zoom(
    scroll: Res<AccumulatedMouseScroll>,
    mut projection: Single<&mut Projection, With<Camera3d>>,
) {
    let Projection::Perspective(perspective) = projection.as_mut() else { return };
    perspective.fov = (perspective.fov - scroll.delta.y * 0.05)
        .clamp(15.0_f32.to_radians(), 90.0_f32.to_radians());
}
```

Чтобы задать угол при создании: добавить к камере `Projection::from(PerspectiveProjection { fov: 70.0_f32.to_radians(), ..default() })`.

08 Свой меш: чанк на экране

> Меш (`Mesh`) — набор вершин и треугольников. Для чанка его строит мешер ядра (массивы чисел), а Bevy превращает массивы в `Mesh` и показывает. Пример собран по `3d/generate_custom_mesh.rs`.

Из массивов в Mesh

```rust
use bevy::{
    asset::RenderAssetUsages,
    mesh::Indices,
    prelude::*,
    render::render_resource::PrimitiveTopology,
};

/// То, что отдаёт мешер ядра (parodia_core): чистые массивы, без Bevy.
pub struct MeshData {
    pub positions: Vec<[f32; 3]>,
    pub normals: Vec<[f32; 3]>,
    pub uvs: Vec<[f32; 2]>,
    pub indices: Vec<u32>,
}

/// Оболочка в игре: массивы → Mesh Bevy.
pub fn to_bevy_mesh(data: MeshData) -> Mesh {
    Mesh::new(
        PrimitiveTopology::TriangleList, // каждые 3 индекса — треугольник
        RenderAssetUsages::default(),    // копия и в памяти, и в видеокарте
    )
    .with_inserted_attribute(Mesh::ATTRIBUTE_POSITION, data.positions)
    .with_inserted_attribute(Mesh::ATTRIBUTE_NORMAL, data.normals)
    .with_inserted_attribute(Mesh::ATTRIBUTE_UV_0, data.uvs)
    .with_inserted_indices(Indices::U32(data.indices))
}
```

- **Грань блока = 4 вершины + 6 индексов** (два треугольника: 0-1-2 и 0-2-3).
- **Порядок вершин — против часовой стрелки**, если смотреть на грань снаружи. Иначе грань «вывернута» и не видна (отсечение задних граней).
- `RenderAssetUsages::default()` — меш хранится и в обычной памяти, и в видеокарте; так Bevy может сам посчитать коробку видимости. Позже для экономии ОЗУ можно оставить только `RENDER_WORLD` — но тогда коробку (`Aabb`) задавать вручную, как в разделе 09.

Показать чанк

```
fn spawn_chunk(
    mut commands: Commands,
    mut meshes: ResMut<Assets<Mesh>>,              // хранилище мешей
    mut materials: ResMut<Assets<StandardMaterial>>,
    asset_server: Res<AssetServer>,
) {
    let data = MeshData { positions: vec![], normals: vec![], uvs: vec![], indices: vec![] };
// ← из мешера
    let mesh = meshes.add(to_bevy_mesh(data)); // add → Handle<Mesh> (ссылка на меш)

    let material = materials.add(StandardMaterial {
        base_color_texture: Some(asset_server.load("textures/blocks.png")),
        perceptual_roughness: 1.0, // матовый, без бликов
        ..default()
    });

    commands.spawn((
        Mesh3d(mesh),
        MeshMaterial3d(material),
        Transform::from_xyz(0.0, 0.0, 0.0), // угол секции в мире
    ));
}
```

Обновить меш после правки блока

Два способа:

```
// 1) заменить содержимое по той же ссылке
meshes.insert(&mesh3d.0, new_mesh).ok();

// 2) создать новый меш и подменить компонент (старый удалится сам, когда на него не останется ссылок)
commands.entity(chunk_entity).insert(Mesh3d(meshes.add(new_mesh)));
```

Bevy сам пересчитывает коробку видимости (`Aabb`) при смене меша — чанк не пропадёт из-за отсечения.

Свет, чтобы было видно

```
commands.spawn((
    DirectionalLight { illuminance: 10_000.0, ..default() }, // «солнце»
    Transform::from_xyz(3.0, 10.0, 2.0).looking_at(Vec3::ZERO, Vec3::Y),
));
```

09 Свой материал и шейдер (WESL)

> Свой материал нужен, когда `StandardMaterial` не подходит: упакованные вершины, цвет по затенению, массив текстур (этапы 7, 10 ТЗ). В Bevy 0.20 шейдеры пишутся на **WESL** — это WGSL с модулями и `import`. Файлы — `assets/shaders/*.wesl`.

Простой материал (по примеру shader/shader_material.rs)

```rust
use bevy::{prelude::*, reflect::TypePath, render::render_resource::AsBindGroup,
shader::ShaderRef};

const SHADER: &str = "shaders/blocks.wesl";

#[derive(Asset, TypePath, AsBindGroup, Debug, Clone)]
pub struct BlockMaterial {
    #[uniform(0)]                   // binding 0 в шейдере
    pub tint: LinearRgba,
    #[texture(1)]                   // binding 1
    #[sampler(2)]                   // binding 2
    pub atlas: Option<Handle<Image>>,
}

impl Material for BlockMaterial {
    fn fragment_shader() -> ShaderRef {
        SHADER.into()
    }
}

// регистрация — без неё материал не нарисуется
app.add_plugins(MaterialPlugin::<BlockMaterial>::default());
```

```wesl
// assets/shaders/blocks.wesl
import bevy_pbr::render::forward_io::VertexOutput;

@group(constants::MATERIAL_BIND_GROUP) @binding(0) var<uniform> tint: vec4<f32>;
@group(constants::MATERIAL_BIND_GROUP) @binding(1) var atlas_texture: texture_2d<f32>;
@group(constants::MATERIAL_BIND_GROUP) @binding(2) var atlas_sampler: sampler;

@fragment
fn fragment(mesh: VertexOutput) -> @location(0) vec4<f32> {
    // свет по стороне грани (раздел 12 карты знаний): верх 1.0, бока 0.8/0.7, низ 0.5
    let n = mesh.world_normal;
    let shade = select(select(0.7, 0.8, abs(n.x) > 0.5), select(0.5, 1.0, n.y > 0.0), abs(n.y)
> 0.5);
    let color = textureSample(atlas_texture, atlas_sampler, mesh.uv) * tint;
    return vec4<f32>(color.rgb * shade, color.a);
}
```

- **Номера `@binding(N)` в шейдере = номера в `#[uniform(N)]`, `#[texture(N)]`, `#[sampler(N)]`** в Rust. Не совпали — ошибка при запуске.

- `import` пишется **без** `#` (старое `#import` — до 0.20). Пути `bevy_pbr::...` — встроенные модули Bevy.
- В буферах **нет** `vec3` — только `vec4`, `vec2`, `f32`, `u32` (выравнивание 16 байт).

Массив текстур (по примеру shader/array_texture.rs)

Плитки блоков — слои одной текстуры: нет «просачивания» соседних плиток, и жадный меш может повторять плитку.

```
use bevy::image::{ImageArrayLayout, ImageLoaderSettings};

let blocks = asset_server
    .load_builder()
    .with_settings(|s: &mut ImageLoaderSettings| {
        // картинка — плитки 16×16 столбиком: N строк = N слоёв
        s.array_layout = Some(ImageArrayLayout::RowCount { rows: 32 });
    })
    .load("textures/blocks_array.png");

#[derive(Asset, TypePath, AsBindGroup, Debug, Clone)]
pub struct BlockArrayMaterial {
    #[texture(0, dimension = "2d_array")]
    #[sampler(1)]
    pub blocks: Handle<Image>,
}
```

В шейдере: `var blocks: texture_2d_array<f32>;` и `textureSample(blocks, blocks_sampler, uv, layer)`, где `layer` — номер плитки.

Свой атрибут вершины: упакованная вершина u32 (этап 7)

Схема по примеру `shader_advanced/custom_vertex_attribute.rs`. Это продвинутый шаг — делать после того, как обычный меш работает.

```rust
use bevy::{
    camera::{primitives::Aabb, visibility::NoAutoAabb},
    mesh::{MeshVertexAttribute, MeshVertexBufferLayoutRef},
    pbr::{MaterialPipeline, MaterialPipelineKey},
    render::render_resource::{RenderPipelineDescriptor, SpecializedMeshPipelineError,
VertexFormat},
};

// «высокий» случайный номер атрибута, чтобы не совпал со встроенными
pub const ATTRIBUTE_PACKED: MeshVertexAttribute =
    MeshVertexAttribute::new("Packed", 917_305_221, VertexFormat::Uint32);

impl Material for PackedBlockMaterial {
    fn vertex_shader() -> ShaderRef { "shaders/packed_blocks.wesl".into() }
    fn fragment_shader() -> ShaderRef { "shaders/packed_blocks.wesl".into() }
    fn enable_prepass() -> bool { false } // без позиций встроенные проходы не сработают
    fn enable_shadows() -> bool { false } // тени — позже, своим проходом
    fn specialize(
        _pipeline: &MaterialPipeline,
        descriptor: &mut RenderPipelineDescriptor,
        layout: &MeshVertexBufferLayoutRef,
        _key: MaterialPipelineKey<Self>,
    ) -> Result<(), SpecializedMeshPipelineError> {
        let vertex_layout = layout.0.get_layout(&[ATTRIBUTE_PACKED.at_shader_location(0)])?;
        descriptor.vertex.buffers = vec![vertex_layout];
        Ok(())
    }
}

// у меша нет ATTRIBUTE_POSITION → Bevy не посчитает коробку сам: даём её вручную
commands.spawn((
    Mesh3d(mesh),
    MeshMaterial3d(material),
    Transform::from_xyz(sx, sy, sz),
    Aabb::from_min_max(Vec3::ZERO, Vec3::splat(32.0)),
    NoAutoAabb,
));
```

```
// assets/shaders/packed_blocks.wesl
import bevy_pbr::render::mesh_functions::{get_world_from_local, mesh_position_local_to_clip};

struct Vertex {
    @builtin(instance_index) instance_index: u32,
    @location(0) packed: u32,
};

struct VertexOutput {
    @builtin(position) clip_position: vec4<f32>,
    @location(0) shade: f32,
};

@vertex
fn vertex(v: Vertex) -> VertexOutput {
    // 6 бит x | 6 бит y | 6 бит z | 3 бита сторона | 2 бита AO | 9 бит текстура
    let x = f32(v.packed & 63u);
    let y = f32((v.packed >> 6u) & 63u);
    let z = f32((v.packed >> 12u) & 63u);
    let face = (v.packed >> 18u) & 7u;
    let ao = f32((v.packed >> 21u) & 3u);

    var out: VertexOutput;
    out.clip_position = mesh_position_local_to_clip(
        get_world_from_local(v.instance_index),
        vec4<f32>(x, y, z, 1.0),
    );
    var face_shades = array<f32, 6>(0.8, 0.8, 1.0, 0.5, 0.7, 0.7);
    out.shade = face_shades[face] * (0.55 + 0.15 * ao);
    return out;
}

@fragment
fn fragment(@location(0) shade: f32) -> @location(0) vec4<f32> {
    return vec4<f32>(vec3<f32>(shade), 1.0);
}
```

Важно: этот раздел — схема на этап 7. Он собран по официальному примеру, но вершинный шейдер со своим форматом — место, где чаще всего бывают ошибки запуска. Делать его, когда обычный меш (раздел 08) уже работает, и при первой ошибке присылать текст.

10 Фоновые задачи

> Генерация и меш чанка — долгие. Если делать их в системе `Update`, игра будет «замирать». Решение — отдать работу в **пул фоновых потоков** (`AsyncComputeTaskPool`), а результат забрать, когда готов. По примеру `async_tasks/async_compute.rs`.

```
use bevy::{
    prelude::*,
    tasks::{futures::check_ready, AsyncComputeTaskPool, Task},
};

/// Задача: в фоне строим меш секции. Висит компонентом на сущности секции.
#[derive(Component)]
struct MeshTask(Task<MeshData>);

fn start_mesh_tasks(mut commands: Commands, need_mesh: Query<(Entity, &SectionPos),
Without<MeshTask>>) {
    let pool = AsyncComputeTaskPool::get();
    for (entity, pos) in need_mesh.iter().take(4) { // лимит: не больше 4 новых задач за кадр
        let pos = *pos;
        let task = pool.spawn(async move {
            // ВНУТРИ — только данные, без Commands/Query/ресурсов Bevy
            build_section_mesh(pos) // функция ядра parodia_core
        });
        commands.entity(entity).insert(MeshTask(task));
    }
}

fn finish_mesh_tasks(
    mut commands: Commands,
    mut meshes: ResMut<Assets<Mesh>>,
    mut tasks: Query<(Entity, &mut MeshTask)>,
) {
    for (entity, mut task) in &mut tasks {
        // check_ready не блокирует: готово — Some(результат), нет — None
        if let Some(data) = check_ready(&mut task.0) {
            commands
                .entity(entity)
                .insert(Mesh3d(meshes.add(to_bevy_mesh(data))))
                .remove::<MeshTask>();
        }
    }
}
```

- **Внутрь задачи данные передаются копией или через** `Arc` (общий указатель «только для чтения» между потоками). Сама задача не может трогать мир Bevy.
- Для чанков удобно: снимок «секция + 6 соседей» → `Arc` → задача.
- **Не использовать** `block_on(poll_once(...))` из старых уроков — это блокирует главный поток (так написано прямо в примере Bevy 0.20).
- Тип `SectionPos` и функция `build_section_mesh` — твои, из ядра.

11 Состояния игры

> Состояние (`States`) — в каком «режиме» игра: меню, загрузка, игра, пауза. Системы можно включать только в нужном состоянии. По примеру `state/states.rs` .

```rust
#[derive(States, Debug, Clone, Copy, Default, Eq, PartialEq, Hash)]
pub enum AppState {
    #[default]
    Menu,
    Loading,
    InGame,
    Paused,
}

app.init_state::<AppState>()
    .add_systems(OnEnter(AppState::Menu), spawn_menu)        // один раз при входе
    .add_systems(OnExit(AppState::Menu), despawn_menu)       // один раз при выходе
    .add_systems(Update, move_player.run_if(in_state(AppState::InGame)));

// перейти в другое состояние
fn start_game(mut next: ResMut<NextState<AppState>>) {
    next.set(AppState::Loading);
}

// узнать текущее
fn print_state(state: Res<State<AppState>>) {
    info!("Сейчас: {:?}", state.get());
}
```

Удобная мелочь: сущность с компонентом `DespawnOnExit(AppState::Menu)` удалится сама при выходе из меню — не нужно писать `despawn_menu` .

12 Текст, F3 и рамка блока

> Интерфейс в Bevy — тоже сущности: `Node` (блок разметки, как div), `Text` (текст). Камера для 3D сама рисует и интерфейс.

Важно: встроенный шрифт Bevy не знает кириллицу. В нём около 95 символов — только латиница; русский текст будет квадратиками. Нужен свой шрифт `.ttf` с кириллицей в `assets/fonts/` (например, свободные DejaVu Sans, PT Sans, Noto Sans) и загрузка через `asset_server.load(...)`.

Окно отладки F3 с FPS

По примеру `ui/text/text.rs`:

```rust
use bevy::{
    diagnostic::{DiagnosticsStore, FrameTimeDiagnosticsPlugin},
    input::common_conditions::input_just_pressed,
    prelude::*,
};

#[derive(Component)]
struct DebugText;

pub struct DebugPlugin;

impl Plugin for DebugPlugin {
    fn build(&self, app: &mut App) {
        app.add_plugins(FrameTimeDiagnosticsPlugin::default()) // считает FPS и время кадра
            .add_systems(Startup, spawn_debug_text)
            .add_systems(Update, (
                update_debug_text,
                toggle_debug.run_if(input_just_pressed(KeyCode::F3)),
            ));
    }
}

fn spawn_debug_text(mut commands: Commands, asset_server: Res<AssetServer>) {
    commands.spawn((
        Text::new("…"),
        TextFont {
            font: asset_server.load("fonts/DejaVuSans.ttf").into(), // шрифт с кириллицей
            font_size: FontSize::Px(16.0),
            ..default()
        },
        Node {
            position_type: PositionType::Absolute,
            top: px(8),
            left: px(8),
            ..default()
        },
        DebugText,
    ));
}

fn update_debug_text(
    diagnostics: Res<DiagnosticsStore>,
    camera: Single<&Transform, With<Camera3d>>,
    mut text: Single<&mut Text, With<DebugText>>,
) {
    let fps = diagnostics
        .get(&FrameTimeDiagnosticsPlugin::FPS)
        .and_then(|d| d.smoothed())
        .unwrap_or(0.0);
    let ms = diagnostics
        .get(&FrameTimeDiagnosticsPlugin::FRAME_TIME)
        .and_then(|d| d.smoothed())
        .unwrap_or(0.0);
    let p = camera.translation;
    text.0 = format!("Кадров/с: {fps:.0}   Кадр: {ms:.1} мс\nКоординаты: {:.1} {:.1} {:.1}",
```

```
p.x, p.y, p.z);
}

fn toggle_debug(mut vis: Single<&mut Visibility, With<DebugText>>) {
    **vis = match **vis {
        Visibility::Hidden => Visibility::Visible,
        _ => Visibility::Hidden,
    };
}
```

- `text.0 = ...` — заменить строку целиком (`Text` — обёртка над `String`).
- `px(8)`, `percent(100)` — размеры в разметке.
- Для доступности (H-07 ТЗ) состояния писать **словами**, а не цветом.

Рамка вокруг блока под прицелом

`Gizmos` (вспомогательная графика) рисует линии каждый кадр без мешей:

```
fn draw_block_highlight(mut gizmos: Gizmos, target: Res<AimedBlock>) {
    if let Some(pos) = target.0 {
        let center = pos.as_vec3() + Vec3::splat(0.5);
        gizmos.cube(Transform::from_translation(center).with_scale(Vec3::splat(1.002)),
Color::WHITE);
    }
}
```

`AimedBlock` — твой ресурс с `Option<IVec3>` (блок под прицелом из луча DDA ядра). Масштаб `1.002` — чтобы рамка не «тонула» в гранях.

13 Ассеты: файлы игры

> Ассет — файл, который игра грузит во время работы: текстура, шрифт, звук, шейдер. Лежат в папке `assets` **рядом с** `Cargo.toml` **пакета игры** (для ParodiaR — смотри раздел «Частые ошибки»).

```
fn load(asset_server: Res<AssetServer>) {
    let tex: Handle<Image> = asset_server.load("textures/blocks.png");   // путь от папки
assets
    let font: Handle<Font> = asset_server.load("fonts/DejaVuSans.ttf");
}
```

- `load` возвращает `Handle` (ссылку) **сразу**, а сам файл грузится в фоне. Пока не загрузился — объект просто не виден.
- Один и тот же путь — одна и та же загрузка, сколько ни вызывай.
- Свои данные (блоки, биомы в `.ron`) на старте проще читать обычным `std::fs::read_to_string` + `ron::from_str` — без системы ассетов.

14 Частые ошибки

Что видишь	Почему	Что сделать
паника «...Message... not initialized» / «resource does not exist»	забыл `add_message::<T>()` или `init_resource::<T>()`	добавить в `build` плагина
ошибка «conflicting access» / «query conflicts» при запуске	два `&mut` одного компонента в одной системе	`Without<...>` во втором запросе (раздел 03)
система с `Single` молча не работает	сущности нет или их больше одной	проверить, что она создаётся ровно одна
свой материал не рисуется	забыл `MaterialPlugin::<МойМатериал>::default()`	добавить плагин
ошибка шейдера `binding`	номера в Rust и `@binding(N)` не совпали	сверить номера
`#import` — ошибка разбора шейдера	старый синтаксис	`import ...;` без `#`, файл `.wesl`
русский текст — квадратики	встроенный шрифт без кириллицы	свой `.ttf` (раздел 12)
текстура размыта	фильтр по умолчанию — сглаживание	`ImagePlugin::default_nearest()`
грань чанка не видна с одной стороны	вершины по часовой стрелке	поменять порядок индексов
«file not found» для ассета	папка `assets` не там, где ищет Bevy	см. ниже
в уроке `Camera3dBundle`, `PbrBundle`, `EventReader`	урок для старой версии Bevy	искать аналог в `cargo doc` или этом справочнике

Где Bevy ищет папку `assets`: при запуске через `cargo run` — рядом с `Cargo.toml` **запускаемого пакета**. В ParodiaR это `crates/parodia_game/assets`. Если хочешь держать `assets` в корне проекта, как в дереве ТЗ, задай путь явно:

```
DefaultPlugins.set(AssetPlugin {
    file_path: "../../assets".into(), // путь от crates/parodia_game до корня
    ..default()
})
```

15 Словарь: имя в коде → что значит

Имя	Что это
`App`	приложение — вся игра
`Plugin`	плагин — кусок приложения со своими системами
`DefaultPlugins`	стандартный набор плагинов (окно, рендер, ввод…)
`Startup`, `Update`, `FixedUpdate`, `Last`	расписания: при старте / каждый кадр / постоянный шаг / в конце кадра
`Entity`	номер сущности
`Component`	компонент — данные на сущности
`Resource`	ресурс — одна вещь на всю игру
`Commands`	очередь изменений мира (создать/удалить)
`spawn`, `despawn`, `insert`, `remove`	создать, удалить, добавить компонент, убрать компонент
`children!`	создать детей сущности
`Query<...>`	запрос к сущностям с нужными компонентами
`Single<...>`	ровно одна сущность
`With<T>`, `Without<T>`, `Changed<T>`	фильтры запроса: есть T / нет T / T изменился
`Res<T>`, `ResMut<T>`	прочитать / изменить ресурс
`Local<T>`	своя переменная системы
`SystemSet`	группа систем для порядка
`.chain()`, `.before()`, `.after()`, `.in_set()`	порядок систем
`.run_if(...)`	запускать при условии
`Message`, `MessageWriter`, `MessageReader`	сообщение, писатель, читатель
`States`, `NextState`, `OnEnter`, `OnExit`, `in_state`	состояния игры
`DespawnOnExit`	удалить сущность при выходе из состояния
`Time`, `Time<Fixed>`, `Timer`	время, постоянный шаг, таймер
`ButtonInput<KeyCode>`	клавиатура (физические клавиши)
`ButtonInput<MouseButton>`	кнопки мыши
`AccumulatedMouseMotion`, `AccumulatedMouseScroll`	движение мыши, колесо за кадр

Имя	Что это
`CursorOptions`, `CursorGrabMode`	видимость и захват курсора
`Transform`	позиция, поворот, масштаб
`Vec2`, `Vec3`, `IVec3`, `Quat`	векторы (f32), вектор целых, кватернион поворота
`Camera3d`, `Projection`, `PerspectiveProjection`	3D-камера, проекция, угол обзора
`DirectionalLight`	направленный свет («солнце»)
`Mesh`, `Mesh3d`	меш и компонент «показать этот меш»
`Indices::U32`	индексы треугольников
`StandardMaterial`, `MeshMaterial3d`	стандартный материал и компонент «этим материалом»
`Material`, `AsBindGroup`, `MaterialPlugin`	свой материал: трейт, связь полей с шейдером, регистрация
`ShaderRef`	путь к файлу шейдера
`MeshVertexAttribute`, `VertexFormat`	свой атрибут вершины и его формат
`Aabb`, `NoAutoAabb`	коробка видимости и «не пересчитывать её»
`Assets<T>`, `Handle<T>`, `AssetServer`	хранилище, ссылка, загрузчик файлов
`Image`, `ImagePlugin`	картинка-текстура, настройки текстур
`AsyncComputeTaskPool`, `Task<T>`, `check_ready`	пул фоновых потоков, задача, «готово?»
`Node`, `Text`, `TextFont`, `px`, `percent`	блок интерфейса, текст, шрифт, размеры
`Visibility`	видно / скрыто
`Gizmos`	быстрые линии для отладки
`DiagnosticsStore`, `FrameTimeDiagnosticsPlugin`	хранилище замеров, плагин FPS
`info!`, `warn!`, `error!`	запись в лог
`..default()`	остальные поля — по умолчанию